MODULE

01

Git basics

What version control is, why every team uses Git, and how to set it up.

YOU WILL LEARN

Why version control	`01_why_version_control.py`
Install & configure	`02_install_and_config.py`
Exercise: first repository	`ex01_first_repo.py`
Interview Q&A	`INTERVIEW.md`

WHY VERSION CONTROL

final_v2_REALLY_final.py

THE PROBLEM

Without version control, people copy files: **app_old.py, app_backup.py, app_final_v2.py**. Nobody knows which is current, what changed, or why, and two people editing the same file overwrite each other.

THE IDEA

Git saves snapshots of the whole project, called commits. Each one records what changed, who, when and why.

PICTURE IT VERSION CONTROL

A photo album of your project

📷 Every **commit** is a photo of the whole project with a caption (the message). You can flip back to any photo, compare two photos, and the team can take photos in parallel on **branches**.

`edit files`
your working folder

→

`git add`
choose what goes in the photo

→

`git commit`
take the photo + caption

→

`git log`
flip through the album

Git is **distributed**: every clone has the whole album, so you can work offline.

TRY IT VERSION CONTROL

Three snapshots of one file

```
$ git init
Initialized empty Git repository in ~/project/.git/
$ echo "price = 10" > shop.py
$ git add shop.py
$ git commit -m "Add shop with a fixed price"
[main (root-commit) f8b0491] Add shop with a fixed price
 1 file changed, 1 insertion(+)
 create mode 100644 shop.py
$ echo "price = 12" > shop.py
$ git commit -qam "Raise the price to 12"
$ git log --oneline
cacef8e Raise the price to 12
f8b0491 Add shop with a fixed price
$ git show HEAD~1:shop.py
price = 10
```

1. Each commit has an ID (a hash) like **f8b0491**.
2. **HEAD~1** = one commit before the current one.
3. Old versions are always there: nothing is lost.

STEP BY STEP GIT VS GITHUB

Git is not GitHub

	GIT	GITHUB / GITLAB / AZURE DEVOPS
What it is	a program on your computer	a website that hosts Git repositories
Works offline?	yes	no
Gives you	commits, branches, history, merges	pull requests, code review, issues, CI/CD
Command	`git commit`	`git push` sends commits there

STEP BY STEP SETUP

Configure Git once per machine

COMMAND	WHY
`git config --global user.name "Your Name"`	stored in every commit
`git config --global user.email "you@example.com"`	links commits to your GitHub account
`git config --global init.defaultBranch main`	new repos start on **main**
`git config --global core.editor "code --wait"`	VS Code for commit messages
`git config --local user.email "you@company.com"`	a different email in ONE repo

> Three levels: **--system** < **--global** (~/.gitconfig) < **--local** (.git/config). The most specific wins.

TRY IT SETUP

Local beats global

```
$ git init
Initialized empty Git repository in ~/project/.git/
$ git config --global user.email
vishal@example.com
$ git config --local user.email "vishal@company.com"
$ git config --show-scope --get-all user.email
global  vishal@example.com
local   vishal@company.com
$ git config user.email
vishal@company.com
```

1. The global value applies everywhere.
2. The local value wins inside this repository.
3. **--show-scope** shows where each value comes from.

COMMON MISTAKE SETUP

Committing with the wrong identity

✗ WRONG

```
# fresh laptop, nothing configured
git commit -m "Fix login"
# Author: acer <acer@LAPTOP-123.local>
```

Your commits show a random machine name and aren't linked to your GitHub profile, so your contributions don't count.

✓ RIGHT

```
git config --global user.name "Vishal"
git config --global user.email "vishal@example.com"
git commit --amend --reset-author --no-edit
```

Configure once per machine. Fix the last commit with **--amend --reset-author**.

IN REAL JOBS GIT

Git on your first day at work

Get the code

```
git clone git@github.com:company/shop.git
```

Set your work identity

```
git config --local user.email "you@company.com"
```

Daily loop

```
git add -p
git commit -m "Fix coupon rounding"
git push
```

Everything is reviewed

```
open a Pull Request -> review -> merge
```

INTERVIEW SAY THIS IN THE INTERVIEW

“What is Git, and why do teams use it?”

- Git is a **distributed version control system**.
- It records snapshots (**commits**) with who, when and why, so you can compare and restore any version.
- Every clone has the **full history**, so you can work offline and every copy is a backup.
- **Branches** let many developers work in parallel and merge their work safely.

> **Follow-up they ask next:** ”Git vs GitHub?” (Git is the tool; GitHub hosts repositories and adds pull requests, reviews and CI).

RECAP REMEMBER THESE

Module 1 in 5 lines

1. A **commit** = a snapshot + who, when, why.
2. Git is **distributed**: every clone has the full history.
3. Git is the tool; GitHub hosts repositories.
4. Set **user.name** and **user.email** once per machine.
5. Config levels: system < global < local.

PRACTICE NOW

Run exercises/ex01_first_repo.py setup and make your first commit.